Módulo de Controle de Acessos — Modelo Físico e Consultas SQL

Disciplina: Banco de Dados

Aluno(a): _____

SGBD: PostgreSQL

1. Modelo conceitual

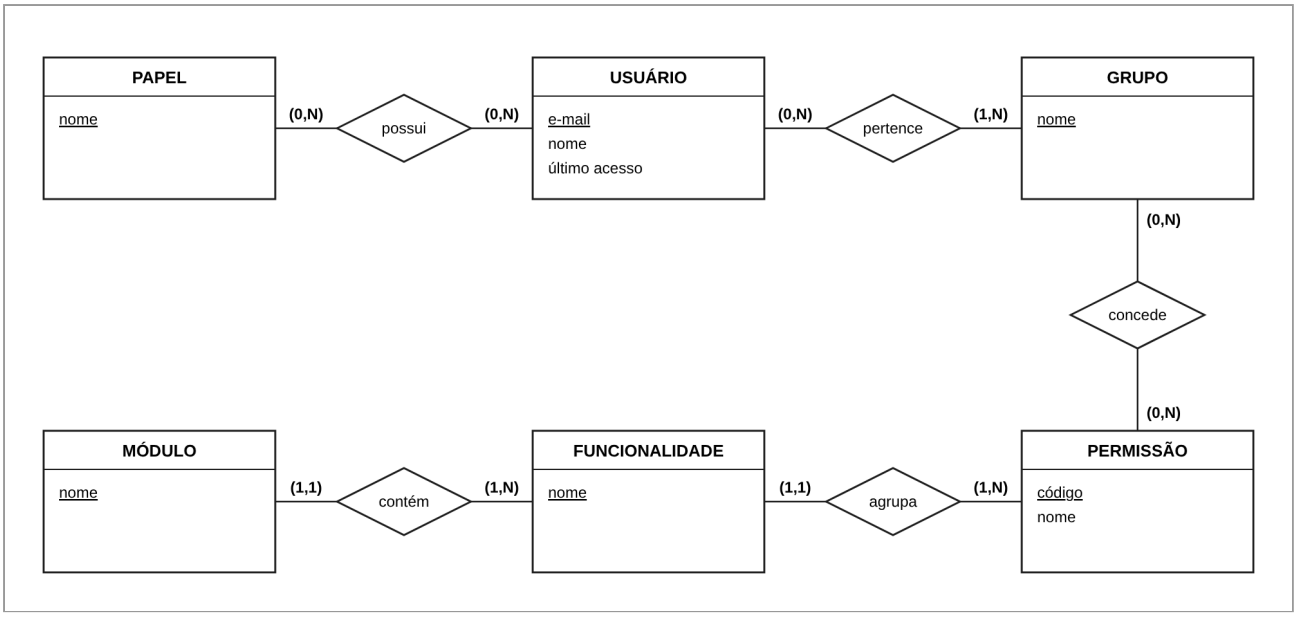


Figura 1 — Diagrama Entidade-Relacionamento

A senha não é guardada no banco, porque o login é feito pelo Active Directory. O nome do usuário é preenchido pela plataforma com o dado vindo do AD; por isso fica nulo até o primeiro acesso.

2. Modelo físico

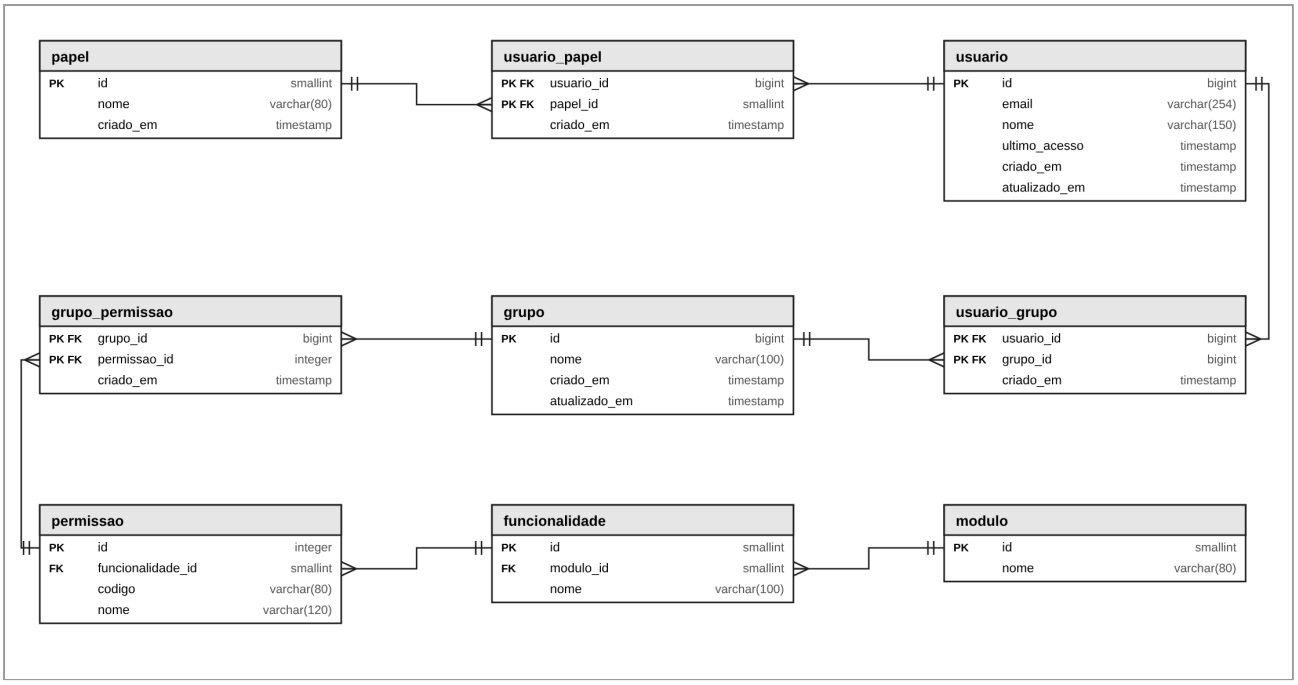


Figura 2 — Modelo físico (PK = chave primária, FK = chave estrangeira)

Relacionamentos

Relacionamento	Cardinalidade	Implementação
USUÁRIO pertence a GRUPO	N:N (usuário com 1 ou mais grupos)	usuario_grupo
USUÁRIO possui PAPEL	N:N (papel opcional)	usuario_papel
GRUPO concede PERMISSÃO	N:N	grupo_permissao
MÓDULO contém FUNCIONALIDADE	1:N	funcionalidade.modulo_id
FUNCIONALIDADE agrupa PERMISSÃO	1:N	permissao.funcionalidade_id

Tabelas, atributos, tipos e restrições

Atributos de negócio vêm do domínio do problema. Atributos técnicos são as chaves substitutas, o código usado pela aplicação e as datas de controle.

usuario

Atributo	Tipo	Chave	Restrições	Categoria
id	bigint	PK	NOT NULL, IDENTITY	Técnico
email	varchar(254)		NOT NULL, CHECK formato, UNIQUE	Negócio
nome	varchar(150)			Negócio
ultimo_acesso	timestamp			Negócio
criado_em	timestamp		NOT NULL, DEFAULT now()	Técnico
atualizado_em	timestamp		NOT NULL, DEFAULT now()	Técnico

papel

Atributo	Tipo	Chave	Restrições	Categoria
id	smallint	PK	NOT NULL, IDENTITY	Técnico
nome	varchar(80)		NOT NULL, UNIQUE	Negócio
criado_em	timestamp		NOT NULL, DEFAULT now()	Técnico

grupo

Atributo	Tipo	Chave	Restrições	Categoria
id	bigint	PK	NOT NULL, IDENTITY	Técnico
nome	varchar(100)		NOT NULL, UNIQUE	Negócio
criado_em	timestamp		NOT NULL, DEFAULT now()	Técnico
atualizado_em	timestamp		NOT NULL, DEFAULT now()	Técnico

modulo

Atributo	Tipo	Chave	Restrições	Categoria
id	smallint	PK	NOT NULL, IDENTITY	Técnico
nome	varchar(80)		NOT NULL, UNIQUE	Negócio

funcionalidade

Atributo	Tipo	Chave	Restrições	Categoria
id	smallint	PK	NOT NULL, IDENTITY	Técnico
modulo_id	smallint	FK	NOT NULL, UNIQUE (modulo_id, nome), FK → modulo.id (RESTRICT)	Negócio
nome	varchar(100)		NOT NULL, UNIQUE (modulo_id, nome)	Negócio

permissao

Atributo	Tipo	Chave	Restrições	Categoria
id	integer	PK	NOT NULL, IDENTITY	Técnico
funcionalidade_id	smallint	FK	NOT NULL, UNIQUE (funcionalidade_id, nome), FK → funcionalidade.id (RESTRICT)	Negócio
codigo	varchar(80)		NOT NULL, UNIQUE	Técnico
nome	varchar(120)		NOT NULL, UNIQUE (funcionalidade_id, nome)	Negócio

usuario_grupo

Atributo	Tipo	Chave	Restrições	Categoria
usuario_id	bigint	PK FK	NOT NULL, FK → usuario.id (CASCADE)	Negócio
grupo_id	bigint	PK FK	NOT NULL, FK → grupo.id (CASCADE)	Negócio
criado_em	timestamp		NOT NULL, DEFAULT now()	Técnico

usuario_papel

Atributo	Tipo	Chave	Restrições	Categoria
usuario_id	bigint	PK FK	NOT NULL, FK → usuario.id (CASCADE)	Negócio
papel_id	smallint	PK FK	NOT NULL, FK → papel.id (RESTRICT)	Negócio
criado_em	timestamp		NOT NULL, DEFAULT now()	Técnico

grupo_permissao

Atributo	Tipo	Chave	Restrições	Categoria
grupo_id	bigint	PK FK	NOT NULL, FK → grupo.id (CASCADE)	Negócio
permissao_id	integer	PK FK	NOT NULL, FK → permissao.id (CASCADE)	Negócio
criado_em	timestamp		NOT NULL, DEFAULT now()	Técnico

Script de criação (DDL)

```
-- Módulo de Controle de Acessos - Modelo físico (PostgreSQL)
-- A senha não é armazenada: o login é feito pelo Active Directory.

CREATE TABLE usuario (
  id             bigint       GENERATED ALWAYS AS IDENTITY,
  email          varchar(254) NOT NULL,
  nome           varchar(150),               -- preenchido com o nome vindo do AD
  ultimo_acesso  timestamp,                  -- NULL = nunca acessou
  criado_em      timestamp    NOT NULL DEFAULT now(),
  atualizado_em  timestamp    NOT NULL DEFAULT now(),
  CONSTRAINT pk_usuario PRIMARY KEY (id),
  CONSTRAINT uq_usuario_email UNIQUE (email),
  CONSTRAINT ck_usuario_email CHECK (email = lower(email) AND email LIKE '%_@_%._%')
);

CREATE TABLE papel (
  id         smallint     GENERATED ALWAYS AS IDENTITY,
  nome       varchar(80)  NOT NULL,
  criado_em  timestamp    NOT NULL DEFAULT now(),
  CONSTRAINT pk_papel PRIMARY KEY (id),
  CONSTRAINT uq_papel_nome UNIQUE (nome)
);

CREATE TABLE grupo (
  id             bigint       GENERATED ALWAYS AS IDENTITY,
  nome           varchar(100) NOT NULL,
  criado_em      timestamp    NOT NULL DEFAULT now(),
  atualizado_em  timestamp    NOT NULL DEFAULT now(),
  CONSTRAINT pk_grupo PRIMARY KEY (id),
  CONSTRAINT uq_grupo_nome UNIQUE (nome)
);

CREATE TABLE modulo (
  id    smallint    GENERATED ALWAYS AS IDENTITY,
  nome  varchar(80) NOT NULL,
  CONSTRAINT pk_modulo PRIMARY KEY (id),
  CONSTRAINT uq_modulo_nome UNIQUE (nome)
);

CREATE TABLE funcionalidade (
  id         smallint     GENERATED ALWAYS AS IDENTITY,
  modulo_id  smallint     NOT NULL,
  nome       varchar(100) NOT NULL,
  CONSTRAINT pk_funcionalidade PRIMARY KEY (id),
  CONSTRAINT uq_funcionalidade_nome UNIQUE (modulo_id, nome),
  CONSTRAINT fk_funcionalidade_modulo FOREIGN KEY (modulo_id)
    REFERENCES modulo (id) ON DELETE RESTRICT
);

CREATE TABLE permissao (
  id                integer      GENERATED ALWAYS AS IDENTITY,
```

```
  funcionalidade_id  smallint     NOT NULL,
  codigo             varchar(80)  NOT NULL,   -- usado pela aplicação (ex.:
EDITAL_VISUALIZAR)
  nome               varchar(120) NOT NULL,   -- texto do toggle
  CONSTRAINT pk_permissao PRIMARY KEY (id),
  CONSTRAINT uq_permissao_codigo UNIQUE (codigo),
  CONSTRAINT uq_permissao_nome UNIQUE (funcionalidade_id, nome),
  CONSTRAINT fk_permissao_funcionalidade FOREIGN KEY (funcionalidade_id)
    REFERENCES funcionalidade (id) ON DELETE RESTRICT
);

-- usuario N:N grupo
CREATE TABLE usuario_grupo (
  usuario_id  bigint    NOT NULL,
  grupo_id    bigint    NOT NULL,
  criado_em   timestamp NOT NULL DEFAULT now(),
  CONSTRAINT pk_usuario_grupo PRIMARY KEY (usuario_id, grupo_id),
  CONSTRAINT fk_usuario_grupo_usuario FOREIGN KEY (usuario_id)
    REFERENCES usuario (id) ON DELETE CASCADE,
  CONSTRAINT fk_usuario_grupo_grupo FOREIGN KEY (grupo_id)
    REFERENCES grupo (id) ON DELETE CASCADE
);

-- usuario N:N papel
CREATE TABLE usuario_papel (
  usuario_id  bigint    NOT NULL,
  papel_id    smallint  NOT NULL,
  criado_em   timestamp NOT NULL DEFAULT now(),
  CONSTRAINT pk_usuario_papel PRIMARY KEY (usuario_id, papel_id),
  CONSTRAINT fk_usuario_papel_usuario FOREIGN KEY (usuario_id)
    REFERENCES usuario (id) ON DELETE CASCADE,
  CONSTRAINT fk_usuario_papel_papel FOREIGN KEY (papel_id)
    REFERENCES papel (id) ON DELETE RESTRICT
);

-- grupo N:N permissao (linha existente = permissão habilitada no grupo)
CREATE TABLE grupo_permissao (
  grupo_id      bigint    NOT NULL,
  permissao_id  integer   NOT NULL,
  criado_em     timestamp NOT NULL DEFAULT now(),
  CONSTRAINT pk_grupo_permissao PRIMARY KEY (grupo_id, permissao_id),
  CONSTRAINT fk_grupo_permissao_grupo FOREIGN KEY (grupo_id)
    REFERENCES grupo (id) ON DELETE CASCADE,
  CONSTRAINT fk_grupo_permissao_permissao FOREIGN KEY (permissao_id)
    REFERENCES permissao (id) ON DELETE CASCADE
);
```

3. Consultas SQL das interfaces

3.1 Login com Active Directory

1) Verificar se o e-mail autenticado no AD está cadastrado na plataforma

```
SELECT id, nome
FROM usuario
WHERE email = 'adailton@positivo.com.br';
```

2) Atualizar o nome vindo do AD e a data do último acesso

```
UPDATE usuario
SET nome = 'Adailton Araújo',
    ultimo_acesso = now(),
    atualizado_em = now()
WHERE id = 1;
```

3) Permissões do usuário logado (soma das permissões de todos os seus grupos)

```
SELECT DISTINCT p.codigo, p.nome
FROM usuario_grupo ug
JOIN grupo_permissao gp ON gp.grupo_id = ug.grupo_id
JOIN permissao p ON p.id = gp.permissao_id
WHERE ug.usuario_id = 1
ORDER BY p.codigo;
```

3.2 Permissões - aba Usuários

4) Listar usuários com grupos, papéis e último acesso (campo "Buscar por usuário")

```
SELECT u.id,
       u.nome,
       u.email,
       (SELECT string_agg(g.nome, ', ' ORDER BY g.nome)
          FROM usuario_grupo ug JOIN grupo g ON g.id = ug.grupo_id
         WHERE ug.usuario_id = u.id) AS grupos,
       (SELECT string_agg(p.nome, ', ' ORDER BY p.nome)
          FROM usuario_papel up JOIN papel p ON p.id = up.papel_id
         WHERE up.usuario_id = u.id) AS papeis,
       u.ultimo_acesso
FROM usuario u
WHERE u.nome ILIKE '%positivo%' OR u.email ILIKE '%positivo%'
ORDER BY u.nome;
```

5) Opções do campo "Grupo" no cadastro de usuário

```
SELECT id, nome FROM grupo ORDER BY nome;
```

6) Opções do campo "Papel" no cadastro de usuário

```
SELECT id, nome FROM papel ORDER BY nome;
```

7) Adicionar usuário (e-mail, grupos e papéis)

```
BEGIN;
INSERT INTO usuario (email) VALUES ('maria.lima@positivo.com.br');

INSERT INTO usuario_grupo (usuario_id, grupo_id)
SELECT u.id, g.id
FROM usuario u, grupo g
WHERE u.email = 'maria.lima@positivo.com.br' AND g.id IN (1, 4);

INSERT INTO usuario_papel (usuario_id, papel_id)
SELECT u.id, p.id
FROM usuario u, papel p
WHERE u.email = 'maria.lima@positivo.com.br' AND p.id IN (2);
COMMIT;
```

8) Carregar usuário para edição

```
SELECT email FROM usuario WHERE id = 6;
SELECT grupo_id FROM usuario_grupo WHERE usuario_id = 6;
SELECT papel_id FROM usuario_papel WHERE usuario_id = 6;
```

9) Salvar edição do usuário

```
BEGIN;
UPDATE usuario
SET email = 'maria.silva@positivo.com.br', atualizado_em = now()
WHERE id = 6;

DELETE FROM usuario_grupo WHERE usuario_id = 6;
INSERT INTO usuario_grupo (usuario_id, grupo_id) VALUES (6, 1), (6, 2);

DELETE FROM usuario_papel WHERE usuario_id = 6;
INSERT INTO usuario_papel (usuario_id, papel_id) VALUES (6, 1);
COMMIT;
```

10) Excluir usuário (os vínculos são removidos por ON DELETE CASCADE)

```
DELETE FROM usuario WHERE id = 6;
```

3.3 Permissões - aba Grupos

11) Listar grupos com quantidade de permissões e de usuários (campo "Buscar por grupo")

```
SELECT g.id,
       g.nome,
       (SELECT count(*) FROM grupo_permissao gp WHERE gp.grupo_id = g.id) AS permissoes,
       (SELECT count(*) FROM usuario_grupo ug WHERE ug.grupo_id = g.id) AS usuarios
FROM grupo g
WHERE g.nome ILIKE '%editais%'
ORDER BY g.nome;
```

12) Usuários de um grupo (ícone da coluna Usuários)

```
SELECT u.nome, u.email
FROM usuario_grupo ug
JOIN usuario u ON u.id = ug.usuario_id
WHERE ug.grupo_id = 4
ORDER BY u.nome;
```

3.4 Adicionar / Editar grupo

13) Aba Permissões: permissões por módulo e funcionalidade, com o estado do toggle

```
SELECT m.nome AS modulo,
       f.nome AS funcionalidade,
       p.id   AS permissao_id,
       p.nome AS permissao,
       (gp.grupo_id IS NOT NULL) AS habilitada
FROM permissao p
JOIN funcionalidade f ON f.id = p.funcionalidade_id
JOIN modulo m ON m.id = f.modulo_id
LEFT JOIN grupo_permissao gp ON gp.permissao_id = p.id AND gp.grupo_id = 4
ORDER BY m.id, f.id, p.id;
```

14) Aba Usuários: usuários com o checkbox marcado se já pertencem ao grupo

```
SELECT u.id,
       u.nome,
       u.email,
       EXISTS (SELECT 1 FROM usuario_grupo x
               WHERE x.usuario_id = u.id AND x.grupo_id = 4) AS marcado,
       (SELECT string_agg(g.nome, ', ' ORDER BY g.nome)
          FROM usuario_grupo ug JOIN grupo g ON g.id = ug.grupo_id
         WHERE ug.usuario_id = u.id) AS grupos
FROM usuario u
ORDER BY u.nome;
```

15) Adicionar grupo com permissões e usuários

```
BEGIN;
INSERT INTO grupo (nome) VALUES ('Análise de propostas');

INSERT INTO grupo_permissao (grupo_id, permissao_id)
SELECT g.id, p.id
FROM grupo g, permissao p
WHERE g.nome = 'Análise de propostas' AND p.id IN (6, 7, 9);

INSERT INTO usuario_grupo (usuario_id, grupo_id)
SELECT u.id, g.id
FROM usuario u, grupo g
WHERE g.nome = 'Análise de propostas' AND u.id IN (2, 4);
COMMIT;
```

16) Salvar edição do grupo (nome, toggles e usuários)

```
BEGIN;
UPDATE grupo
SET nome = 'Análise de propostas e editais', atualizado_em = now()
WHERE id = 6;

DELETE FROM grupo_permissao WHERE grupo_id = 6;
INSERT INTO grupo_permissao (grupo_id, permissao_id) VALUES (6, 6), (6, 7), (6, 9), (6,
11);

DELETE FROM usuario_grupo WHERE grupo_id = 6;
INSERT INTO usuario_grupo (usuario_id, grupo_id) VALUES (2, 6), (4, 6);
COMMIT;
```

17) Excluir grupo (permissões e vínculos são removidos por ON DELETE CASCADE)

```
DELETE FROM grupo WHERE id = 6;
```